Vivado Simulation Guide

Everything you need to know to simulate
VHDL designs with the Vivado simulator (XSim)

TeraNova – FPGA Acceleration

Author	Kai Janssen (4994817)
Version	5 November 2026
Tools	Vivado 2025.1.1 (Vivado simulator, XSim), Windows
Companion	*Rung Simulation Guide* (the ladder in Vivado)

Based on the Vivado Design Suite User Guide: Logic Simulation (UG900, version 2025.1), the Vivado Tcl Command Reference (UG835) and hands-on experience with the TeraNova testbenches. References at the end.

Contents

1 How to use this guide **3**

2 The concepts in ten minutes **3**

2.1 What a simulation is . . . 3
2.2 The three steps . . . 3
2.3 Kinds of simulation . . . 4
2.4 Projects, filesets and simulation sets . . . 4
2.5 Libraries . . . 4
2.6 The run folder . . . 4

3 Practice run: a counter from start to finish **5**

3.1 Create a project . . . 5
3.2 Add the design and the testbench . . . 5
3.3 Set the run time and run . . . 5
3.4 Look at the result . . . 6
3.5 Make it fail on purpose . . . 6

4 Writing testbenches that work in XSim **6**

4.1 The skeleton . . . 6
4.2 Clock and reset . . . 7
4.3 Waiting . . . 7
4.4 Self-checking: report and assert . . . 7
4.5 Ending the simulation . . . 8
4.6 Reading and writing files . . . 8
4.7 Delta cycles and races . . . 8
4.8 VHDL in XSim 2025.1: language versions . . . 8

5 Simulation settings **9**

5.1 Hierarchy update and compile order . . . 10

6 The simulator workspace **10**

6.1 The simulation toolbar . . . 11

7 The wave window **11**

7.1 What gets recorded . . . 11
7.2 Adding and arranging signals . . . 11
7.3 Radix (number format) . . . 11
7.4 Analog view . . . 12
7.5 Navigating . . . 12
7.6 Cursors, markers and measuring time . . . 12

7.7 Saving and reopening 12

8 Debugging **13**
8.1 Breakpoints and stepping 13
8.2 Reading and forcing values 13
8.3 U, X and Z 13

9 Simulating without the GUI **13**

10 IP, block designs and HLS (for later) **14**

11 Making simulations faster **14**

12 Troubleshooting **15**

13 Cheat sheet **16**

A Practice files **16**

References **18**

1 How to use this guide

This guide teaches the Vivado simulator from zero. It is written for the situation "I have tried simulating in Vivado before and it never worked". Read it in this order:

1. Section 2: what a simulation actually is in Vivado (three tools, three steps). Ten minutes, and most past problems become understandable.
2. Section 3: a complete practice run with a small counter. Do it once with your hands on the keyboard; it takes about 20 minutes.
3. Sections 4–8: reference material, to read when you need it: writing testbenches, settings, the wave window, debugging.
4. Section 12: the troubleshooting table. When something fails, start here.
5. Section 13: a one-page cheat sheet.

The practice files are in `C:\PLD\sim\vivado\practice`. The improvement-ladder simulation for the tutor meeting has its own guide (*Rung Simulation Guide*).

Important. Simulation needs **no licence**. The node-locked licence on Badr's laptop is only needed for synthesis and implementation of the ZCU102 design. Behavioural simulation with XSim runs on any PC with Vivado 2025.1.1 installed.

2 The concepts in ten minutes

2.1 What a simulation is

A simulation runs your HDL code on the PC instead of on the FPGA. A *testbench* (an extra HDL file with no ports) creates the clock, the reset and the input stimulus, instantiates your design as the *unit under test* (UUT) and, ideally, checks the outputs. The simulator calculates every signal over time; you look at the result in the *wave window* and in the messages of the Tcl Console.

2.2 The three steps

Every Vivado simulation, from the GUI or from a script, consists of three separate programs (Figure 1). When "the simulation fails", it always fails in one of these three, and the message tells you which one. Knowing this is the single most useful thing in this guide.

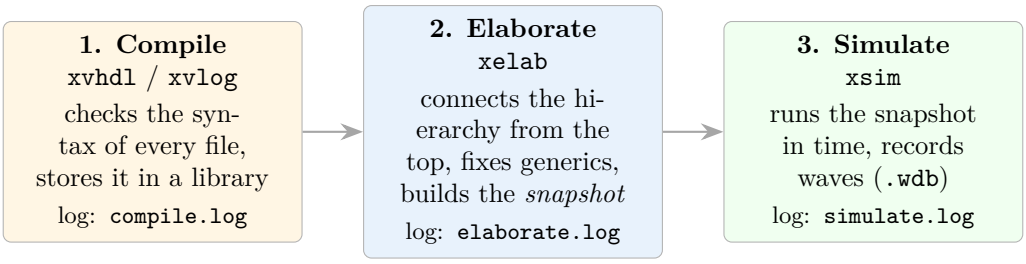


Figure 1: The three steps behind Run Simulation. The logs are in `<project>.sim\sim_1\behav\xsim`.

- **Compile** errors (`[VRFC 10-...]`) are language errors in a file: a typo, a missing library clause, VHDL-2008 code compiled as plain VHDL.
- **Elaborate** errors are about how the files fit together: the top cannot be found, a component has no matching entity, a port does not exist, a generic has the wrong type. In the

GUI they appear as `[USF-XSim-62] 'elaborate' step failed`; the real message is in `elaborate.log`.

- **Simulate** problems are about behaviour: the run stops too early, signals stay `U` or `X`, an assertion fires, a file cannot be opened.

2.3 Kinds of simulation

Kind	What is simulated
Behavioural (RTL)	Your HDL source code as written. Fast, full visibility. **This is what you use 95 % of the time, and what this guide is about.**
Post-synthesis functional	The synthesised netlist, without delays. Checks that synthesis understood your code.
Post-implementation timing	The placed and routed netlist with real delays (SDF). Slow; rarely needed when the design meets timing in static timing analysis. Needs the licensed implementation.

2.4 Projects, filesets and simulation sets

In a Vivado project the Sources window has three kinds of filesets:

- **Design Sources** (`sources_1`): what is synthesised.
- **Constraints** (`constrs_1`): XDC files; not used by simulation.
- **Simulation Sources** (`sim_1`, `sim_2`, ...): testbenches and simulation-only models. A simulation set normally also *includes* the design sources, so you do not add your design twice.

A testbench belongs in *Simulation Sources* only, so it is never synthesised. Every simulation set has its own **top** (shown in bold in the Sources window). You can have several simulation sets, for example one per testbench; the *active* one (right-click Make Active) is the one Run Simulation uses.

2.5 Libraries

Every compiled VHDL unit lives in a *library*. In a project the default library is `xil_defaultlib`, and the VHDL name `work` always means "the library this file is compiled into". You rarely change this, but it is the clean way to have two different versions of an entity with the same name in one simulation: put each version in its own library (the ladder simulation does exactly this). AMD's own libraries (`unisim`, `xpm`, the IP libraries) come precompiled for XSim; `compile_simlib` is only needed for third-party simulators such as Questa.

2.6 The run folder

The GUI writes everything for a behavioural run of `sim_1` into

```
<project>.sim\sim_1\behav\xsim\
```

The simulator also *runs* in this folder. Two consequences:

- the three logs (`compile.log`, `elaborate.log`, `simulate.log`) and the scripts Vivado generated (`compile.bat`, `elaborate.bat`, `simulate.bat`, `*.prj`) are here: the first place to look when something fails;

- a relative file name in VHDL (`file_open(f, "data.txt", ...)`) is relative to **this folder**, not to your source folder. This is the most common reason why a testbench that reads a file "cannot find" it (Section 4.6).

3 Practice run: a counter from start to finish

This exercise uses two small files that are already on your PC: `C:\PLD\sim\vivado\practice\counter.vhd` (the design) and `C:\PLD\sim\vivado\practice\tb_counter.vhd` (the testbench). Both are listed in Appendix A.

3.1 Create a project

1. Start **Vivado 2025.1.1**. Quick Start > Create Project > Next.
2. Project name `practice_sim`, location for example `C:\PLD_tmp`. Tick Create project subdirectory. Next.
3. RTL Project, tick Do not specify sources at this time. Next.
4. Part: any device is fine for simulation. Pick for example a small Zynq-7000 or keep the default. Next > Finish.
5. Settings (Flow Navigator, top) > General: set Target language to `VHDL`. Under Settings > Simulation the Target simulator must be `Vivado Simulator` and Simulator language `VHDL` or `Mixed` (defaults are fine). OK.

3.2 Add the design and the testbench

1. Flow Navigator PROJECT MANAGER > Add Sources > Add or create design sources > Next > Add Files: select `counter.vhd`. Leave Copy sources into project *unticked* (the project then uses your file in place). Finish.
2. Add Sources again, now Add or create simulation sources. Simulation set `sim_1`. Add Files: `tb_counter.vhd`. Finish.
3. In the Sources window select both files; in the Source File Properties window set Type to **VHDL 2008**. (Or Tcl: `set_property file_type {VHDL 2008} [get_files *.vhd]`.)
4. Open Simulation Sources > sim_1. `tb_counter` must be shown in **bold** with the counter below it as `uut`. If not: right-click `tb_counter` > Set as Top.

Why. The testbench has no ports, so Vivado could never use it as a synthesis top; but if it is added as a *design* source, synthesis would try. And if the top of `sim_1` is the design instead of the testbench, the simulation runs, but nothing drives the inputs: every signal is `U`. Both are classic "my simulation does not work" causes.

3.3 Set the run time and run

1. Flow Navigator SIMULATION > Simulation Settings > Simulation tab: `xsim.simulate.runtime` is **1000ns** by default. The counter test needs about 2.7 µs. Type `-all` (run until the testbench ends) and click OK.
2. SIMULATION > Run Simulation > Run Behavioral Simulation.

3. The status bar shows the three steps; then the simulator workspace opens.

Important. The default run time of 1000 ns is the reason many first simulations "do nothing": the run stops long before anything interesting happens. Either set the run time to `-all` (and make the testbench end by itself, Section 4.5) or press Run All after the launch.

3.4 Look at the result

1. The **Tcl Console** (bottom) shows `Note: TEST PASSED` and the time when the testbench reported it.
2. The **wave window** shows the top-level signals of the testbench. Click Zoom Fit in the wave toolbar.
3. Right-click `count` > Radix > Unsigned Decimal. Now the values read 0, 1, 2, ...
4. In the **Scope** window click `uut`; the **Objects** window now shows the signals inside the counter (`cnt`). Drag `cnt` into the wave window. It is empty: it was not recorded during the run.
5. Click Restart (toolbar), then Run All. Now `cnt` has a full waveform.

That last point explains the second classic problem: the simulator records only the signals that are in the wave window (or explicitly logged) *while it runs*. Section 7.1 shows how to log everything from the start.

3.5 Make it fail on purpose

Learning to read failures is part of the exercise:

1. Change `if unsigned(count) /= 10` into `/= 11` in the testbench and save. Click Relaunch Simulation (the circular arrow in the simulation toolbar). The console now shows an `Error:` line with the time; the testbench ends with `TEST FAILED`.
2. Remove the semicolon after `en <= '1'` and relaunch. Now the *compile* step fails: a pop-up says the step failed, and the Tcl Console and `compile.log` show a `[VRFC 10-...]` message with the file and line number.
3. Undo both changes and relaunch until it passes again.

4 Writing testbenches that work in XSim

4.1 The skeleton

Every testbench in this project follows the same pattern (see `tb_counter.vhd`):

```
entity tb_x is end entity;                      -- no ports
architecture sim of tb_x is
  signal clk : std_logic := '0';                -- ALWAYS give an initial value
  signal finished : boolean := false;
begin
  clk <= not clk after 5 ns when not finished; -- 100 MHz clock, stops at the end
  uut : entity work.x port map (...);           -- direct entity instantiation
```

```
  process                                          -- stimulus + checks, no sensitivity
    list
  begin
    rst <= '1'; wait for 100 ns; wait until rising_edge(clk); rst <= '0';
    ...                                            -- drive inputs, wait, check outputs
    finished <= true;                              -- stops the clock -> simulation ends
    wait;                                          -- never resume
  end process;
end architecture;
```

4.2 Clock and reset

- `clk <= not clk after 5 ns` only works if `clk` starts at `'0'`: `not 'U'` is `'U'` forever. A clock that stays `U` is the third classic problem.
- Change inputs right *after* a rising edge (`wait until rising_edge(clk); x <= ...;`). The design then samples them at the next edge, like real synchronous logic, and there are no races.
- To check an output, wait for the edge and then a little more (`wait for 1 ns;`) or check on the falling edge. Reading a register output in the same delta cycle as the edge gives the *old* value (Section 4.7).

4.3 Waiting

`wait for 100 ns;`	a fixed time
`wait until rising_edge(clk);`	the next rising edge
`wait until done = '1';`	until a signal *changes* to a value (if it is already `'1'`, it waits for the next change!)
`wait until done = '1' for 1 ms;`	the same with a timeout: always use one, so a hung design does not hang the simulation
`wait on a, b;`	until `a` or `b` changes
`wait;`	forever (end of a process)

4.4 Self-checking: report and assert

Do not rely on looking at waves to decide whether a test passed. Let the testbench check and print:

```
assert result = expected
  report "result " & integer'image(to_integer(result)) & " /= expected" severity error
    ;
report "TEST PASSED";                              -- severity note is the default
```

Severity `note` and `warning` only print; `error` prints an `Error:` line and *continues* (XSim default); `failure` stops the simulation. The Tcl Console shows every message with its simulation time, which is often the quickest way to find where in the wave window to look.

Tip. For tables and clean output use `textio` instead of `report`: `write(l, string'("x = ") & integer'image(x)); writeline(output, l);` prints exactly that line, without the time and file information that `report` adds. The ladder testbench prints its summary table this way.

4.5 Ending the simulation

With run time `-all` the simulator runs until nothing is left to do. A free-running clock never stops, so the simulation would run forever. Three ways to end it:

1. **Stop the clock** (`when not finished`, as above). When all processes wait forever, the simulation ends by itself. Simple and portable; used in all TeraNova testbenches.
2. **VHDL-2008** `std.env.stop;` or `std.env.finish;` (`use std.env.all;`). `stop` pauses the simulator like a breakpoint, `finish` ends the run. XSim's behaviour at a finish can be set with the `xsim -onfinish quit|stop` option.
3. **A fixed run time** (`run 2 us`). Fine for a quick look, but then the testbench does not decide when it is done.

4.6 Reading and writing files

`std.textio` works in XSim. Remember that relative file names are relative to the run folder (`...\sim_1\behav\xsim`). Options, from most to least robust:

- generate the data inside VHDL (a function or constant), so no file is needed (done in the ladder testbench);
- use an absolute path with forward slashes: `"C:/PLD/sim/data.txt"`;
- copy the file into the run folder (and remember that Reset Simulation deletes it).

Files written by the testbench (results, logs) also appear in the run folder.

4.7 Delta cycles and races

A signal assignment never takes effect immediately: the new value appears one *delta cycle* later, at the same simulation time. Most strange simulation results come from this:

- a process woken by `rising_edge(clk)` that reads a register output sees the value *before* the edge;
- creating a second clock with a signal assignment (`clk2 <= clk;`) delays it by one delta, which can make data cross in the wrong cycle in simulation only; drive every flip-flop from the same clock signal;
- a combinational loop never settles: the simulation "hangs" at one time. XSim can stop it with `xsim -maxdeltaid <n>`.

4.8 VHDL in XSim 2025.1: language versions

- Set the file type of every VHDL-2008 file to **VHDL 2008**. Without it the file is compiled with the older standard, and 2008 features (`process(all)`, conditional assignments inside processes, `std.env`, reading `out` ports, `?=`, ...) give compile errors. The command line equivalent is `xvhdl -2008`.
- Vivado 2025.1 adds **VHDL 2019** support (new section in UG900 2025.1): file type `VHDL 2019` (`set_property FILE_TYPE {VHDL 2019} [get_files x.vhd]`). Supported 2019 features listed: conditional expressions, conditional return, empty records, mode views for interfaces.
- The 2025.1 revision history of UG900 notes that the separate "-93 mode" support was removed. Do not rely on VHDL-93-only behaviour; set the file type explicitly.
- **Project lesson**: the RTL that Vitis HLS 2025.1 generated for `RAM_INTERFACE` contains one VHDL-2019 conditional constant (`constant ... := a when c else b`) in its `m_axi` adapter. In `C:\PLD\sim\hls_rtl` that line was rewritten as an equivalent VHDL-2008 function call (marked `SIM PATCH`). In XSim either use those patched copies as VHDL

2008, or compile the original file as VHDL 2019.
- XSim does not support a few rarely used VHDL constructs (UG900, *VHDL Language Support Exceptions*): guarded blocks and guarded signal assignments, `postponed` processes and statements, the `unaffected` waveform, `linkage` ports and only partially `buffer` ports, aggregates as case choices, concurrent procedure calls that contain a `wait`, and some alias forms. None of these is used in the TeraNova sources.

Important. VHDL is not case sensitive. A variable `n` in a process hides a constant `N` from a package; the code compiles and simulates without any warning, but uses the wrong value. This exact bug happened while the ladder testbench was written (a loop over `N*N` result words ran on forever because `n` was the cycle counter). Give counters and constants clearly different names.

5 Simulation settings

Flow Navigator > SIMULATION > Simulation Settings (or Settings > Simulation). The settings belong to a simulation set; the Tcl names are shown because you can set them in a script with `set_property -name <name> -value <v> -objects [get_filesets sim_1]`.

Setting (Tcl name)	What it does / when you need it
Top of the dialog	
Simulation top module name	The testbench to simulate. Same as Set as Top. Tcl: `set_property top tb_x [get_filesets sim_1]`.
Compilation tab	
Generics/Parameters	Override generics of the top: `LAT=100`. Tcl: `set_property generic {LAT=100} [get_filesets sim_1]`.
`xsim.compile.xvhdl.relax`	Relaxed language checks (on by default in projects).
`xsim.compile.xvhdl.nosort`	Do not sort VHDL files (use the order of the fileset).
`xsim.compile.xvhdl.more_options`	Extra `xvhdl` options.
`xsim.compile.tcl.pre`	Tcl file run before compilation.
Elaboration tab	
`xsim.elaborate.debug_level`	`typical` (default in projects: waves, breakpoints, line tracing), `all`, `off`. With `off` you cannot see waves.
`xsim.elaborate.relax`	Relaxed checks during elaboration.
`xsim.elaborate.mt_level`	Parallel compile jobs (`auto`).
`xsim.elaborate.rangecheck`	Run-time range checks for VHDL (slower, catches out-of-range integers).
`xsim.elaborate.xelab.more_options`	Extra `xelab` options.
Simulation tab	
`xsim.simulate.runtime`	How long to run after launch. Default `1000ns`. `-all`: until the testbench ends. Empty: load only and wait for you.
`xsim.simulate.log_all_signals`	Record *every* signal from time 0, so anything you add to the wave window later has its full history. Costs memory and speed in big designs.
`xsim.simulate.tcl.post`	A Tcl file run at the end of the launch (for example: add signals to the wave window).

Setting (Tcl name)	What it does / when you need it
`xsim.simulate.custom_tcl`	A Tcl file that *replaces* the generated run script.
`xsim.simulate.wdb`	Name of the waveform database file.
`xsim.simulate.saif`, `saif_scope`	Switching activity for power analysis.
`xsim.simulate.xsim.more_options`	Extra `xsim` options.
`xsim.simulate.no_quit`	Do not quit at the end of a batch run.

5.1 Hierarchy update and compile order

Vivado normally works out the hierarchy and compile order itself (Sources right-click > Hierarchy Update):

- Automatic Update and Compile Order (default);
- Automatic Update, Manual Compile Order: you set the order in the Compile Order tab (Tcl: `set_property source_mgmt_mode DisplayOnly [current_project]`);
- No Update, Manual Compile Order (`None`).

Use manual order only when the automatic order goes wrong, for example with several entities of the same name in different libraries.

6 The simulator workspace

After a launch, the Flow Navigator shows SIMULATION with the open simulation. The windows:

Window	Use
Scope	The design hierarchy (instances, processes, generate blocks). Selecting a scope fills the Objects window. Right-click Add to Wave Window adds all signals of that scope.
Objects	The signals, ports, constants and variables of the selected scope, with their current value. Drag them into the wave window.
Wave	The waveforms (Section 7). Several wave windows (wave configurations) can be open as tabs.
Tcl Console	Every action as a Tcl command, all messages (`report`, `assert`, errors), and a prompt to type commands.
Sources	In simulation mode it shows the simulated hierarchy; double-click opens the source, where you can set breakpoints.
Messages	Errors and warnings of compile/elaborate, with links to the line.

6.1 The simulation toolbar

Button	**Tcl** and meaning
Restart	`restart` – back to time 0, keeps the wave window and the snapshot (no recompile).
Run All	`run -all` – run until the testbench ends or a breakpoint is hit.
Run For *(time box)*	`run 10 us` – run a fixed time further.
Step	`step` – execute one HDL statement (debugging).
Break	stop a running simulation (the pause button).
Relaunch Simulation	`relaunch_sim` – recompile and re-elaborate after you changed a source, keep the wave window.

Important. After editing a VHDL file, use Relaunch Simulation, not Restart. Restart reuses the old compiled snapshot, so your change is not in it. (UG900 also advises against using Run Simulation again to re-run: it may close the current simulation and lose wave window changes.) To start completely clean: SIMULATION > right-click Run Simulation > Reset Behavioral Simulation.

7 The wave window

7.1 What gets recorded

The simulator writes values to the waveform database (`.wdb`) only for objects that are *logged*: everything in the wave window, plus what you log with `log_wave`. A signal added after the run has no history until you Restart and run again. Three ways around it:

- add the signals first, then Restart + Run All;
- type `log_wave -r /` right after the launch (logs everything) and then run;
- set `xsim.simulate.log_all_signals` to `true` in the settings.

Useful `log_wave` forms (UG900): `log_wave -r /` (whole design), `log_wave /tb/uut/*` (one scope), `log_wave [get_objects -filter {type == signal} /tb/uut/*]` (internal signals only). VHDL *variables* are not logged; make a signal copy if you want to see one.

7.2 Adding and arranging signals

- Drag from the Objects window, or right-click a scope > Add to Wave Window. Tcl: `add_wave /tb/uut/cnt`, `add_wave -r /tb/uut` (recursive), `add_wave -radix unsigned -name "count" /tb/count`.
- **Groups**: select signals, right-click > New Group. Tcl: `set g [add_wave_group "Writer"]; add_wave -into $g /tb/x`. Groups collapse with the triangle.
- **Dividers**: right-click > New Divider; Tcl `add_wave_divider "AXI"`.
- **Virtual bus**: combine single bits into one bus (New Virtual Bus).
- **Rename**: right-click > Rename, or Name > Short/Long to hide the hierarchy path.
- **Colour**: right-click > Signal Color.
- **Reverse bit order**: right-click > Reverse Bit Order.

7.3 Radix (number format)

Right-click a bus > Radix: Default, Binary, Hexadecimal, Octal, ASCII, Unsigned Decimal, Signed Decimal, Signed Magnitude, Real. In Tcl the values are `bin`, `hex`, `oct`, `dec`, `unsigned`, `ascii`: `add_wave -radix hex /tb/addr` or `set_property radix unsigned [get_waves`

`count]`. A radix set in the Objects window does not change the wave window, and the reverse. VHDL `integer` signals are always shown as decimal numbers, and **enumerated types** (state machines) are shown by their state names, which makes `state` signals very readable.

7.4 Analog view

A bus can be drawn as a curve: right-click > Waveform Style > Analog, then Analog Settings for the range and interpolation. Nice for counters, filter outputs or a growing cycle counter.

7.5 Navigating

Zoom Fit	show the whole run (toolbar). The first thing to press after a run.
Zoom in/out	toolbar, or **Ctrl + mouse wheel**; drag a rectangle in the time ruler to zoom to it.
Scroll	mouse wheel scrolls the signal list; the scroll bar at the bottom moves in time.
Go to start / end	toolbar buttons.
Previous / next transition	toolbar arrows: jump the cursor to the next edge of the selected signal. Fastest way to step through handshakes.
Find value	search a bus for a value with the find function of the wave window (UG900: *Searching a Value in Waveform Configuration*).

7.6 Cursors, markers and measuring time

- **Main cursor**: click in the waveform area. The *Value* column shows every signal's value at the cursor time; the time is shown at the top of the cursor.
- **Markers**: put the cursor on an edge, right-click > Markers > Add Marker. Markers stay when the cursor moves; drag them to move. Tcl: `add_wave_marker -name "done" 1012 us`.
- **Measuring**: place a marker on the first event (for example the rising edge of `start`) and move the cursor to the second. The time between them is shown as the distance between cursor and marker; with Snap to Transition (on by default) both snap exactly to edges. The Floating Ruler (in the wave window settings) shows times relative to the cursor.
- Cycles instead of nanoseconds: divide by the clock period (10 ns at 100 MHz), or let the testbench count cycles in an `integer` signal and read it in the Value column.

7.7 Saving and reopening

- **Wave configuration** (`.wcfg`): which signals, groups, radixes, markers. File > Simulation Waveform > Save Configuration As, Tcl `save_wave_config x.wcfg`. Add the `.wcfg` to the simulation set, and it opens automatically next time.
- **Waveform database** (`.wdb`): the recorded values. Saved automatically in `<project>.sim\sim_1\behav\xsim`. A finished run can be opened later without simulating again: Flow > Open Static Simulation, or from a command prompt `xsim <name>.wdb -gui`.
- To keep a run for a presentation, copy the `.wdb` and the `.wcfg` somewhere safe: a new launch overwrites them.

8 Debugging

8.1 Breakpoints and stepping

- Open a source from the simulation Sources window. Click next to a line number: a red dot is a breakpoint. Run All stops there; the Value column and hovering show the values at that moment. Tcl: `add_bp tb_counter.vhd 45`, `remove_bps -all`.
- Step executes one statement, also into other processes; good to understand the order in which processes run.
- **Conditions**: stop or print when an expression becomes true: `add_condition {/tb/uut/tc == 1} {stop}`; instead of `stop` any Tcl can run, for example `{puts "tc at [current_time]"}`.

8.2 Reading and forcing values

- `get_value /tb/uut/cnt`, `get_value -radix hex /tb/addr` – the value now.
- `report_drivers /tb/x` – who drives this signal (essential for `X`: two drivers).
- `add_force /tb/rst 1`, `add_force /tb/clk {0 0ns} {1 5ns} -repeat_every 10ns`, `remove_forces -all`. Or right-click a signal > Force Constant / Force Clock. Forces are good for quick experiments, not for real tests: put stimulus in the testbench.

8.3 U, X and Z

`U` (uninitialised)	Never assigned. Testbench input not driven, wrong top, clock without initial value, or reset never applied.
`X` (conflict)	Two drivers with different values (`report_drivers`), or arithmetic on a `U`/`X` value.
`Z` (high impedance)	Not connected (an output port left open, an input port of the UUT not mapped).

Find the *first* moment the bad value appears (go to time 0, step with "next transition"), then look at what drives it.

9 Simulating without the GUI

The same three steps can be run from a command prompt; this is how the ladder batch script works. Open Start > Vivado 2025.1.1 Tcl Shell (or run `C:\Xilinx\2025.1.1\Vivado\settings64.bat` in a `cmd` window), go to an empty folder and:

```
xvhdl -2008 C:/PLD/sim/vivado/practice/counter.vhd C:/PLD/sim/vivado/practice/
    tb_counter.vhd
xelab -debug typical -s tb_counter_snap work.tb_counter
xsim tb_counter_snap -R                 :: run to the end, text only
xsim tb_counter_snap -gui               :: or: open the GUI with this snapshot
```

`xvhdl -2008 -work lib f.vhd`	compile as VHDL 2008 into library `lib` (default `work`); `-relax`; `-prj files.prj` for a list (`vhdl2008 xil_defaultlib f.vhd` per line)
`xelab -debug typical`	elaborate with wave visibility (**default is `off`**: no waves!)
`xelab -generic_top "LAT=100"`	override a generic of the top
`xelab -s name lib.top`	name of the snapshot and the top unit
`xelab -R` / `xsim -R`	run to the end immediately
`xsim -tclbatch run.tcl`	run Tcl commands (`log_wave -r /`, `run -all`, `quit`)
`xsim -wdb x.wdb`, `-view x.wcfg`	database name / wave configuration to open (with `-gui`)
`xsim -log f.log`, `-onfinish stop`	log name / what to do at a finish

`launch_simulation -scripts_only` (or `export_simulation -simulator xsim`) writes the scripts a project would run, which is a good way to see exactly what the GUI does.

10 IP, block designs and HLS (for later)

- **IP from the catalog**: after Generate Output Products each IP has a simulation model; projects add it automatically. Many IPs have an Open IP Example Design with a ready testbench.
- **Block designs**: create the HDL wrapper and instantiate it in a testbench. A design with the Zynq UltraScale+ processing system needs AMD's verification IP (VIP) to drive the AXI ports from the testbench; the PS itself is not simulated. For TeraNova it is simpler and faster to simulate the PL data path on its own with AXI memory models, as in `C:\PLD\sim`.
- **Vitis HLS**: C/RTL co-simulation runs XSim on the generated RTL. The generated VHDL/Verilog (`<component>\hls\impl\vhdl`) can be used in your own XSim testbench, as `C:\PLD\sim\hls_rtl` does.
- **AXI transactions**: XSim can show AXI interfaces of IP-integrator designs as transaction bars (UG900, *Analyzing AXI Interface Transactions*). For plain RTL, add the handshake signals (`valid`, `ready`, `last`) in a group and step with "next transition".

11 Making simulations faster

- Log only what you need. `log_all_signals` on a big design writes a large `.wdb`.
- Do not print every cycle; count in variables and print once at the end.
- Keep `debug_level` at `typical`; `all` is slower.
- Large arrays as VHDL *variables* in a process are faster than signals and are not logged (the TeraNova memory models do this).
- Simulate only the part of the design you are interested in (the PL data path, not the whole block design).

12 Troubleshooting

Symptom	Likely cause	Fix
Simulation stops at 1000 ns, nothing happened	default run time	`xsim.simulate.runtime = -all`, or Run All
All signals `U`	top of `sim_1` is the design, not the testbench; or clock without initial value	Set as Top on the testbench; `signal clk : std_logic := '0'`
Signal in the wave window is empty / starts late	added after the run	Restart + Run All, or `log_wave -r /` first, or `log_all_signals`
Change in the source has no effect	used Restart	Relaunch Simulation
Popup "compile step failed", `[VRFC 10-...]`	syntax/language error	file and line in the Tcl Console or `compile.log`; VHDL-2008 code needs file type VHDL 2008
`[USF-XSim-62] 'elaborate' step failed`	hierarchy problem: unknown entity, wrong port, wrong generic, wrong top	read `elaborate.log` in `...\behav\xsim`
"cannot find design unit"	file not in the simulation set, wrong library, or compile order	add the file to `sim_1`; check the library; check Compile Order
File cannot be opened (textio)	relative path, run folder is `...\behav\xsim`	absolute path with `/`, or generate the data in VHDL
Simulation never ends	clock never stops, `wait until` on a signal that never changes	stop the clock; use `wait until x = '1' for <timeout>`; press Break
Time does not advance (hangs)	combinational loop / delta-cycle loop	Break, look at which process runs (`xsim -maxdeltaid`)
Error linking/writing the snapshot, files locked	previous simulation still open, or antivirus scanning `xsim.dir`	Close Simulation; close stale `xsimk.exe` in Task Manager; keep project paths short and without spaces
Testbench shown under Design Sources	added as design source	move it: right-click > Move to Simulation Sources
Wrong values only in simulation	delta cycle race (derived clock, reading right at the edge)	one clock signal; check after the edge (`wait for 1 ns`)
A loop or condition uses a strange value	name clash, VHDL is not case sensitive (`n` vs `N`)	rename; check with `report integer'image(N)`

13 Cheat sheet

Project (Vivado Tcl Console)	
`launch_simulation`	= Run Behavioral Simulation (`-simset sim_2`, `-mode post-synthesis -type functional`)
`relaunch_sim`	recompile + rerun after a source change
`close_sim`	close the simulation
`set_property top tb [get_filesets sim_1]`	testbench as top
`set_property file_type {VHDL 2008} [get_files *.vhd]`	VHDL 2008
`set_property generic {LAT=100} [get_filesets sim_1]`	generic of the top
`set_property -name xsim.simulate.runtime -value -all -objects [get_filesets sim_1]`	run to the end
Running	
`run -all` / `run 10 us` / `restart` / `step`	run to the end / for a time / to time 0 / one statement
`current_time`	where am I
Waves	
`log_wave -r /`	record everything from now on
`add_wave -radix hex /tb/uut/addr`	add with radix
`add_wave_group "X"`, `add_wave -into $g ...`	groups
`add_wave_divider "AXI"`	divider
`add_wave_marker -name m 100 us`	marker
`save_wave_config f.wcfg` / `open_wave_config f.wcfg`	layout
Debug	
`get_value -radix unsigned /tb/x`	read a value
`report_drivers /tb/x`	who drives it
`add_bp file.vhd 42` / `remove_bps -all`	breakpoints
`add_condition {/tb/done == 1} {stop}`	conditional stop
`add_force /tb/x 1` / `remove_forces -all`	force
Command line	
`xvhdl -2008 f.vhd` → `xelab -debug typical -s s work.tb` → `xsim s -R`	the three steps

A Practice files

counter.vhd

```vhdl
-- counter.vhd -- practice design for the Vivado simulation guide
-- 8-bit counter with enable; 'tc' is high for one cycle when the count wraps 255 -> 0
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity counter is
  port (
    clk   : in  std_logic;
    rst   : in  std_logic;                    -- synchronous, active high
    en    : in  std_logic;
    count : out std_logic_vector(7 downto 0);
    tc    : out std_logic);                   -- terminal count
```

```
end entity;

architecture rtl of counter is
  signal cnt : unsigned(7 downto 0) := (others => '0');
begin
  process(clk)
  begin
    if rising_edge(clk) then
      tc <= '0';
      if rst = '1' then
        cnt <= (others => '0');
      elsif en = '1' then
        cnt <= cnt + 1;
        if cnt = 255 then
          tc <= '1';
        end if;
      end if;
    end if;
  end process;
  count <= std_logic_vector(cnt);
end architecture;
```

tb_counter.vhd

```
-- tb_counter.vhd -- self-checking testbench for counter.vhd (VHDL 2008)
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tb_counter is                              -- a testbench has no ports
end entity;

architecture sim of tb_counter is
  constant T_CLK : time := 10 ns;                 -- 100 MHz
  signal clk      : std_logic := '0';
  signal rst      : std_logic := '1';
  signal en       : std_logic := '0';
  signal count    : std_logic_vector(7 downto 0);
  signal tc       : std_logic;
  signal finished : boolean := false;
begin
  -- 1. clock: runs until the stimulus sets 'finished'
  clk <= not clk after T_CLK / 2 when not finished;

  -- 2. the unit under test
  uut : entity work.counter
    port map (clk => clk, rst => rst, en => en, count => count, tc => tc);

  -- 3. stimulus and checks
  process
    variable errors : integer := 0;
  begin
    rst <= '1';
    wait for 3 * T_CLK;
    wait until rising_edge(clk);
    rst <= '0';
```

```
    -- count 10 times, check the value
    en <= '1';
    for i in 1 to 10 loop
      wait until rising_edge(clk);
    end loop;
    wait for 1 ns;                                  -- let the outputs settle after the
    edge
    if unsigned(count) /= 10 then
      report "count is " & integer'image(to_integer(unsigned(count))) & ", expected 10
    " severity error;
      errors := errors + 1;
    end if;

    -- pause: the value must hold
    en <= '0';
    for i in 1 to 5 loop wait until rising_edge(clk); end loop;
    wait for 1 ns;
    assert unsigned(count) = 10 report "count changed while en = '0'" severity error;

    -- run to the wrap-around and check the terminal count pulse
    en <= '1';
    wait until tc = '1' for 300 * T_CLK;
    if tc /= '1' then
      report "no terminal count pulse" severity error;
      errors := errors + 1;
    end if;

    if errors = 0 then
      report "TEST PASSED";
    else
      report "TEST FAILED with " & integer'image(errors) & " error(s)" severity error;
    end if;
    finished <= true;                               -- 4. stop the clock: simulation ends
    wait;
  end process;
end architecture;
```

References

1. AMD, *Vivado Design Suite User Guide: Logic Simulation* (UG900), version 2025.1. https://docs.amd.com/r/2025.1-English/ug900-vivado-logic-simulation
2. AMD, UG900 2025.1, *Vivado Simulator Simulation Options*. https://docs.amd.com/r/2025.1-English/ug900-vivado-logic-simulation/Vivado-Simulator-Simulation-Options
3. AMD, UG900 2025.1, *Vivado Simulator Elaboration Options* and *Compilation Options*. https://docs.amd.com/r/2025.1-English/ug900-vivado-logic-simulation/Vivado-Simulator-Elaboration-Options
4. AMD, UG900 2025.1, *xelab, xvhdl, and xvlog xsim Command Options* and *xsim Executable Options*. https://docs.amd.com/r/2025.1-English/ug900-vivado-logic-simulation/xsim-Executable-Options
5. AMD, UG900 2025.1, *VHDL 2008 Support* / *Compiling and Simulating*, *Targeting VHDL 2019 for a Specific File*, *VHDL Language Support Exceptions*, *Revision History*.
6. AMD, UG900, *Using the log_wave Tcl Command*, *Using Markers*, *Changing the Radix on Individual Objects*, *Saving Simulation Results*, *Re-running the Simulation After Design*

Changes (relaunch), *Adding a Simulation Start-up Script File*.
7. AMD, *Vivado Design Suite Tcl Command Reference Guide* (UG835): `add_wave`, `add_wave_group`, `add_wave_marker`, `launch_simulation`. https://docs.amd.com/r/en-US/ug835-vivado-tcl-commands/add_wave